

Operating Systems and Concurrency

Processes 1

COMP2007

Dan Marsden

(Chao Chen)

{Chao.Chen, Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham

United Kingdom

2026

Recap

Last Lecture

- **Interrupts:**

- Change the normal flow of computation
- Switch from user to kernel mode

- **System calls:**

- Allow us to run kernel code to access services of the operating system
- Can be implemented in terms of interrupts

Goals for Today

Overview

- Introduction to **processes** and their **implementation**
- Process **states** and state **transitions**
- **System calls** for process management
- **The fork and exec pattern**

Processes

Definition

- **A process is an abstraction of a running instance of a program**

Processes

Definition

- **A process is an abstraction of a running instance of a program**
 - A program is **passive** - it's just a file somewhere

Processes

Definition

- **A process is an abstraction of a running instance of a program**
 - A program is **passive** - it's just a file somewhere
 - A process has **control structures** associated with it, may be **active**, and may have **resources** assigned to it (e.g. I/O devices, memory, processor)

Processes

Definition

- **A process is an abstraction of a running instance of a program**
 - A program is **passive** - it's just a file somewhere
 - A process has **control structures** associated with it, may be **active**, and may have **resources** assigned to it (e.g. I/O devices, memory, processor)
- All the information necessary to administer a process is stored by the kernel in a **process control block (PCB)**.

Processes

Definition

- **A process is an abstraction of a running instance of a program**
 - A program is **passive** - it's just a file somewhere
 - A process has **control structures** associated with it, may be **active**, and may have **resources** assigned to it (e.g. I/O devices, memory, processor)
- All the information necessary to administer a process is stored by the kernel in a **process control block (PCB)**.
- All the process control blocks are recorded in the **process table** - typically an array of PCBs.

Processes

Definition

- **A process is an abstraction of a running instance of a program**
 - A program is **passive** - it's just a file somewhere
 - A process has **control structures** associated with it, may be **active**, and may have **resources** assigned to it (e.g. I/O devices, memory, processor)
- All the information necessary to administer a process is stored by the kernel in a **process control block (PCB)**.
- All the process control blocks are recorded in the **process table** - typically an array of PCBs.
- Every process has a unique **process identifier (PID)** used to associate it with its **PCB** - typically this is the corresponding array index.

Processes

Memory Image of Processes

- A **process' memory image** contains:
 - The program code (could be shared between multiple processes running the same code)
 - A data segment, stack and heap
- Every process has its own **logical address space**, in which the **stack** and **heap** are placed at **opposite sides** to allow them to grow

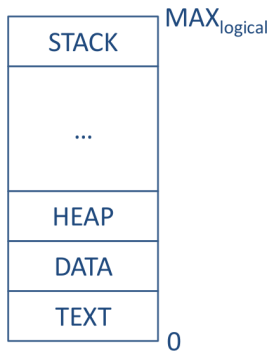
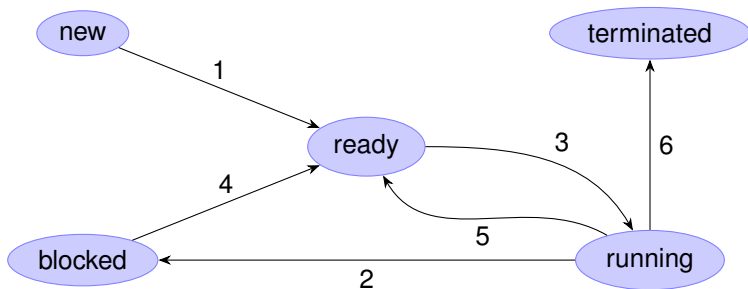


Figure: Representation of a process in memory

Process States and Transitions



Process States and Transitions

States

- A **new** process has just been created. It has a PCB and is waiting to be admitted, although it may not yet be in memory.
- A **ready** process is waiting for CPU to become available.
- A **running** process is currently having its instructions executed by the CPU.
- A **blocked** process cannot continue, e.g. is waiting for I/O
- A **terminated** process is no longer executable. The data structures - PCB - may be temporarily preserved.

Process States and Transitions

States

- A **new** process has just been created. It has a PCB and is waiting to be admitted, although it may not yet be in memory.
- A **ready** process is waiting for CPU to become available.
- A **running** process is currently having its instructions executed by the CPU.
- A **blocked** process cannot continue, e.g. is waiting for I/O
- A **terminated** process is no longer executable. The data structures - PCB - may be temporarily preserved.
- A **suspended** process is swapped out (not discussed further)

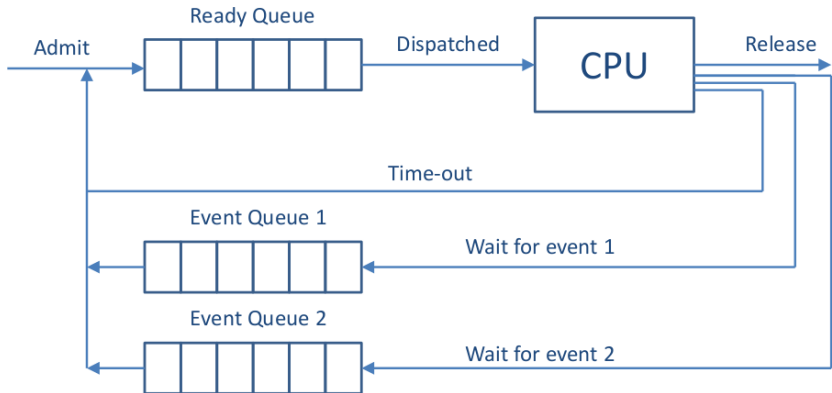
Process States and Transitions

Transitions

- State transitions include: important!!!
 - ① **New** → **ready**: admit the process and commit to execution
 - ② **Running** → **blocked**: e.g. process is waiting for input or carried out a system call
 - ③ **Ready** → **running**: the process is selected by the process scheduler
 - ④ **Blocked** → **ready**: event happens, e.g. I/O operation has finished
 - ⑤ **Running** → **ready**: the process surrenders the CPU, for example due to an **interrupt** or by **pause** give up
 - ⑥ **Running** → **terminated**: process has finished, e.g. program ended or exception encountered
- Interrupts and system calls drive these transitions.

Process States and Transitions

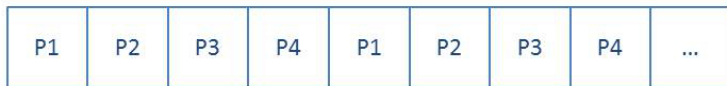
OS Queues



Context Switching

Multi-programming

- Modern computers are multi-programming systems
- Assuming a single processor system, the instructions of individual processes are executed **sequentially**
 - Multi-programming is achieved by interleaving the execution of processes, dividing the CPU time into time-slices
 - Control is exchanged between processes via a procedure known as **context switching**
 - A **trade-off** exists between the length of the **time-slice** and the **context switch time**
 - True parallelism requires hardware support



TIME

Context Switching

Multi-programming (Cont'd)

- When a **context switch** takes place, the system **saves the state** of the old process and **loads the state** of the new process (creates **overhead**)
 - **Saved** $\Rightarrow$ the process control block is **updated**
 - **(Re-)started** $\Rightarrow$ the process control block is **read**

赭生诳

Context Switching

Multi-programming (Cont'd)

Terminology

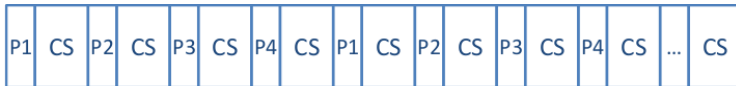
- The **response time** of a process is the ^{used} elapsed time from its creation before a process first gets access to the CPU
- The **effective utilisation** is the fraction of time the CPU is doing useful work - i.e. not context switching

Context Switching

Multi-programming (Cont'ed)

Short time slices result in **good response times** but **low effective utilisation**. For example, assume both context switches and time slices take $1ms$. Then:

- It will take $99 \times (1 + 1) = 198ms$ for the last of 100 processes to start running.
- $\frac{1}{1+1} = 0.5$ of the CPU time is doing useful work.



TIME

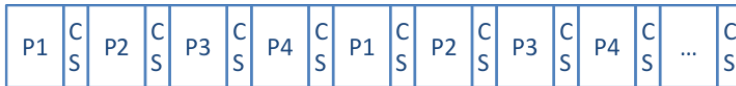


Context Switching

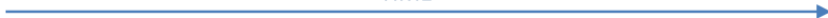
Multi-programming (Cont'd)

Long time slices result in **poor response times** but **better effective utilisation**. For example, assume context switches take $1ms$ and time slices are $100ms$. Then:

- It will take $99 \times (100 + 1) = 9999ms$ for the last of 100 processes to start running.
- $\frac{100}{1+100} = 0.99$ of the CPU time is doing useful work



TIME



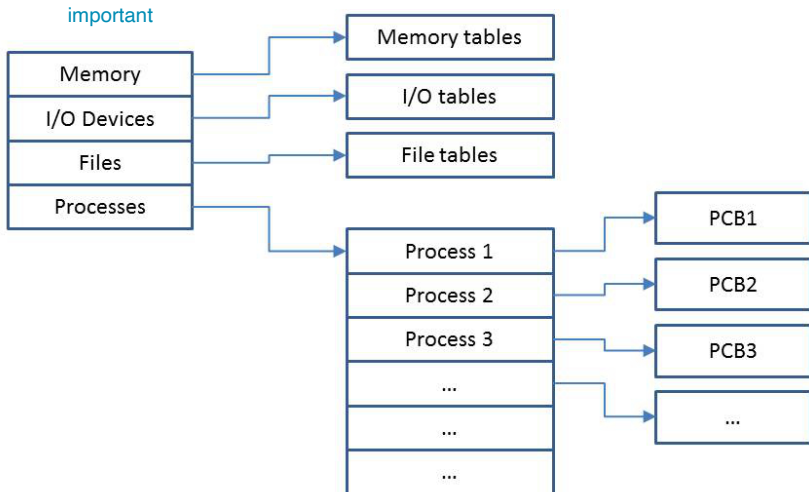
Context Switching

Multi-programming (Cont'ed)

- A **process control block** contains three types of **attributes**:
 - Process identification (PID, UID, Parent PID)
 - Process control information (process state, scheduling information, etc.)
 - Process state information (user registers, program counter, stack pointer, program status word, memory management information, files, etc.)
- **Process control blocks** are kernel data structures, i.e. they are **protected** and only accessible in **kernel mode**!
 - Allowing user applications to access them directly could compromise their integrity
 - The operating system manages them on the user's behalf through system calls (e.g. to set **process priority**)

Process Implementation

Tables and Control Blocks



Process Implementation

Tables and Control Blocks

- An operating system **maintains information** about the status of “resources” in **tables**
 - **Process tables** (process control blocks)
 - **Memory tables** (memory allocation, memory protection, virtual memory)
 - **I/O tables** (availability, status, transfer information)
 - **File tables** (location, status)
- The **process table** holds a **process control block** for each process, allocated upon **process creation**
- Tables are maintained by the **kernel** and are usually **cross referenced**

Context Switching

To switch between processes the kernel will:

Context Switching

To switch between processes the kernel will:

- 1 Save process state (PC, registers, MMU, ...) to PCB.

Context Switching

To switch between processes the kernel will:

- 1 Save process state (PC, registers, MMU, ...) to PCB.
- 2 Update PCB state (running $\mapsto$ ready / blocked / terminated).

Context Switching

To switch between processes the kernel will:

- 1 Save process state (PC, registers, MMU, ...) to PCB.
- 2 Update PCB state (running $\mapsto$ ready / blocked / terminated).
- 3 Record PID in appropriate queue (ready / blocked).

Context Switching

To switch between processes the kernel will:

- 1 Save process state (PC, registers, MMU, ...) to PCB.
- 2 Update PCB state (running $\mapsto$ ready / blocked / terminated).
- 3 Record PID in appropriate queue (ready / blocked).
- 4 Run scheduler, select new PID from ready queue.

Context Switching

To switch between processes the kernel will:

- 1 Save process state (PC, registers, MMU, ...) to PCB.
- 2 Update PCB state (running $\mapsto$ ready / blocked / terminated).
- 3 Record PID in appropriate queue (ready / blocked).
- 4 Run scheduler, select new PID from ready queue.
- 5 Update PCB state (ready $\mapsto$ running).

Context Switching

To switch between processes the kernel will:

- 1 Save process state (PC, registers, MMU, ...) to PCB.
- 2 Update PCB state (running $\mapsto$ ready / blocked / terminated).
- 3 Record PID in appropriate queue (ready / blocked).
- 4 Run scheduler, select new PID from ready queue.
- 5 Update PCB state (ready $\mapsto$ running).
- 6 Restore state from PCB (PC, registers, MMU, ...).

Context Switching

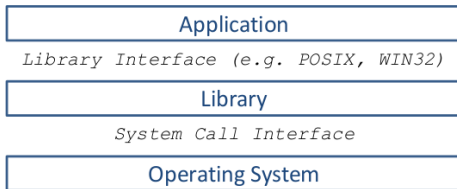
To switch between processes the kernel will:

- 1 Save process state (PC, registers, MMU, ...) to PCB.
- 2 Update PCB state (running $\mapsto$ ready / blocked / terminated).
- 3 Record PID in appropriate queue (ready / blocked).
- 4 Run scheduler, select new PID from ready queue.
- 5 Update PCB state (ready $\mapsto$ running).
- 6 Restore state from PCB (PC, registers, MMU, ...).
- 7 Return control to running the new process.

System Calls

Process Creation

- The true system calls are “**wrapped**” in the **OS libraries** following a well defined interface
- For example, on Unix-like operating systems `fork` is called to create a copy of a process. On Linux, the underlying system call used to implement `fork` is `clone`.



System Calls

Process Termination

- System calls via `exit` and `abort` can be used to explicitly notify the OS that the **process has terminated**
 - Resources must be de-allocated
 - Output must be flushed
 - Process admin may have to be carried out
- A system calls to terminate other processes:
 - UNIX/Linux: `kill()`
 - Windows: `TerminateProcess()`

System Calls

Waiting for Processes

- It is possible to **wait for a process and recover information about how it terminated**, using system calls such as `waitpid` to wait for a chosen PID.
- The operating system **must retain information about terminated processes to service wait calls**.
- Once a wait call has been serviced, the remaining control data can be cleaned up.

Process Creation in Linux

Fork

- fork() creates an **exact copy** of the current process
 - The first instruction carried out by the child is the first one after the `fork` call
- `fork()` returns the **process identifier (PID)** of the child process to the **parent process**.
- `fork()` **returns 0** to the **child process**.

Process Creation in Linux

The Fork and Exec Pattern

A common pattern is the following sequence:

- 1 Call `fork()` to create an exact copy of the current process.
- 2 In the child process call one of the “exec” functions to replace the current process with a new program.

Process Creation in Linux

The Fork and Exec Pattern

A common pattern is the following sequence:

- 1 Call `fork()` to create an exact copy of the current process.
- 2 In the child process call one of the “exec” functions to replace the current process with a new program. E.g. `exec1("/bin/ls", "ls", "-l", 0)`
replaces the current process with ls,

Process Creation in Linux

The Fork and Exec Pattern

A common pattern is the following sequence:

- 1 Call `fork()` to create an exact copy of the current process.
- 2 In the child process call one of the “exec” functions to replace the current process with a new program. E.g. `exec1("/bin/ls", "ls", "-l", 0)` replaces the current process with `ls`,

This is a typical pattern of calls in a Unix shell such as `bash`.

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```


Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```


Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Processes

Process Creation in Linux

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
(class) pid_t const pid = fork();
    if(pid < 0){
        printf("Fork failure\n");
        return -1;
    } else if(pid == 0) {
        printf("Child process\n");
        execl("/bin/ls", "ls", "-l", 0);
    } else {
        int status = 0;
        waitpid(pid, &status, 0);
        printf("Child %d returned %d\n", pid, status);
    }
}
```

Test your understanding

- Why might you run `fork` without running a subsequent `exec`?
- Do you always need to call `exit` to end a process?
- Why does a process control block contain data about register contents?
- Why might it be useful to retain a process control block for a terminated process?

1. The child process can continue executing the same program as the parent without loading a new program.
2. No. A process also terminates when `main()` returns or reaches its end.
3. The OS must save register contents during a context switch so the process can later resume correctly.
4. To preserve termination information, such as the exit status, until the parent collects it with `wait()` or `waitpid()`.

Processes 1 - Core Revision Summary

process abstraction · PCB / process table · states and transitions · context switching · process system calls · fork / exec / wait

source-marked / annotated content

1. Topic Overview

This lecture introduces a **process** as the operating-system abstraction for a running program, then follows its complete lifecycle: how the kernel represents it, how it moves between states, how context switching lets many processes share a CPU, and how Unix/Linux system calls create, replace, wait for, and terminate processes.

Lecture logic: program vs process → PCB / process table → memory image → states / queues → context switching → system calls → fork() + exec() + waitpid().

2. Core Concepts

Process, program, PCB and PID

- ▶ ♦ A **process is an abstraction of a running instance of a program**; a program itself is passive - just a file.
- ▶ A process may be active and has control structures and resources such as **memory, I/O devices and processor time**.
- ▶ ♦ The kernel stores the information needed to administer a process in a **Process Control Block (PCB)**.
- ▶ PCBs are held in the **process table**; each process has a unique **PID** used to associate it with its PCB.

Process memory image

- ▶ A process memory image contains **program code (text), data, heap and stack**; code may be shared by processes running the same program.
- ▶ ♦ Every process has its own **logical address space**. Stack and heap are placed at opposite sides so both can grow.

Process states

- ▶ ♦ **New**: created; PCB exists; waiting to be admitted. **Ready**: waiting for CPU.
- ▶ ♦ **Running**: CPU is executing its instructions. **Blocked**: cannot continue, e.g. waiting for I/O.
- ▶ ♦ **Terminated**: no longer executable; some control data / PCB may be temporarily retained.
- ▶ **Suspended**: swapped out; mentioned but not discussed further in this lecture.

PCB contents and protection

A PCB contains three broad groups of attributes:

- ▶ **Identification**: PID, UID, parent PID.
- ▶ **Control information**: process state, scheduling information, etc.
- ▶ **State information**: registers, PC, stack pointer, PSW, MMU / memory information, files, etc.
- ▶ ♦ PCBs are **kernel data structures**; user code cannot access them directly because that could compromise their integrity. The OS exposes controlled operations via system calls.

OS resource tables

The kernel maintains cross-referenced **process, memory, I/O and file tables**. The process table contains one PCB for each process and is allocated when the process is created.

3. Key Mechanisms / Algorithms

1) Process-state transitions

New → Ready → Running → Terminated Running → Blocked → Ready

1. **New → Ready**: admit the process and commit to execution.
2. **Running → Blocked**: wait for input / event or perform a system call that cannot complete immediately.
3. **Ready → Running**: selected by the process scheduler.
4. **Blocked → Ready**: awaited event occurs, e.g. I/O finishes.
5. **Running → Ready**: process gives up / loses CPU, e.g. interrupt or pause.
6. **Running → Terminated**: program finishes or encounters an exception.

♦ **Interrupts and system calls drive these transitions.** The ready queue and event queues hold processes waiting for CPU or events.

2) Multiprogramming and the time-slice trade-off

On a single processor, individual instructions execute sequentially. **Multiprogramming** interleaves processes by dividing CPU time into **time-slices**; switching control between them is a **context switch**. True parallelism requires hardware support.

- ▶ ♦ **Short time-slices**: better response time, but more context-switch overhead and lower effective utilisation.
- ▶ ♦ **Long time-slices**: poorer response time, but less relative switching overhead and higher effective utilisation.

3) Context switch - kernel sequence

1. ♦ Save old process state (PC, registers, MMU, ...) to its PCB.
2. Update old PCB state: running → ready / blocked / terminated.
3. Record its PID in the appropriate queue where applicable.
4. Run scheduler; choose a new PID from the ready queue.
5. Update selected PCB: ready → running.
6. Restore its state from PCB (PC, registers, MMU, ...).
7. Return control to the newly running process.

A context switch therefore creates **overhead**: it performs bookkeeping rather than useful application work.

4) Unix/Linux process creation: fork - exec - wait

Parent process → fork() → child copy → exec(...) replaces child program
Parent → waitpid(childPID, ...) → resumes after child termination

- ▶ ♦ **fork()** creates a copy of the current process. The child begins at the instruction immediately after the **fork()** call.
- ▶ ♦ In the **parent**, **fork()** returns the child's PID; in the **child**, it returns **0**. A negative value indicates failure in the lecture example.
- ▶ ♦ An **exec** function **replaces the current process image with a new program**; e.g. `execl("/bin/ls", "ls", "-l", 0)`.
- ▶ **waitpid(pid, &status, 0)** waits for the chosen child and retrieves termination information. The OS must retain enough data about a terminated process until the wait is serviced.
- ▶ This **fork + exec** pattern is typical of Unix shells such as **bash**.

4. Formula / Rules / Conditions

Response time and effective utilisation

- ▶ **Response time:** elapsed time from process creation until it first gets CPU access.
- ▶ **Effective utilisation:** fraction of CPU time doing useful work rather than context switching.

For the lecture's simple repeated pattern with time-slice length T and context-switch time C :

$$\text{effective utilisation} = \frac{T}{T + C}$$

With 100 processes, the last process first runs after $99(T + C)$ in the lecture example.

- ▶ ♦ $T = 1 \text{ ms}, C = 1 \text{ ms}$: response for last process = $99(1 + 1) = 198 \text{ ms}$; utilisation = $1/(1 + 1) = 0.5$.
- ▶ ♦ $T = 100 \text{ ms}, C = 1 \text{ ms}$: response = $99(100 + 1) = 9999 \text{ ms}$; utilisation = $100/(100 + 1) \approx 0.99$.

fork / exec / termination rules

- ▶ `pid_t const pid = fork();` `pid_t` is the process-ID type; here `pid` stores the **return value of `fork()`**, not automatically the current process's own PID.
- ▶ ♦ Child-side `pid == 0` is an identity signal from `fork()`; the child's **real PID is not 0**. Its actual PID can be obtained with `getpid()`.
- ▶ `exec()` does **not create another process**; it replaces the calling process's program while the process identity continues.
- ▶ A process does **not always need an explicit `exit()`**: returning from or reaching the end of `main()` also terminates it.
- ▶ `exit/abort` explicitly notify the OS of termination; resources must be deallocated, output flushed, and process administration performed. Other processes can be targeted using e.g. Unix/Linux `kill()`.

5. Comparison & Common Confusions

Concept A	Concept B	Key Difference / Revision Point
Program	Process	Program is passive code/file; process is a running instance with state and resources.
Ready	Blocked	Ready can run once CPU is available; blocked cannot proceed until an event occurs.
Multiprogramming	True parallelism	Multiprogramming can interleave work on one CPU; true parallelism requires hardware support.
Short time-slice	Long time-slice	Short improves response but increases relative switching overhead; long does the reverse.
PCB	Process memory image	PCB is kernel control/state metadata; memory image contains code, data, heap and stack.
<code>fork()</code> return in parent	<code>fork()</code> return in child	Parent receives child PID; child receives 0. Neither statement means the child's real PID is 0.
<code>fork()</code>	<code>exec()</code>	<code>fork()</code> creates a process copy; <code>exec()</code> replaces the calling process image with another program.
Terminated process	Fully cleaned-up process	Terminated process may retain control data until a parent services <code>wait/waitpid</code> .

6. Exam / Revision Focus

Must Know

- ▶ Definitions: **process, PCB, PID, process table, ready/running/blocked/terminated, context switch, response time, effective utilisation**.
- ▶ ♦ The **five main process states and all six transitions**; know that interrupts/system calls drive transitions.
- ▶ The **7-step context-switch sequence** and why PCB stores register / PC / MMU state.
- ▶ `fork()` return rules, `exec()` replacement semantics, and `waitpid()` purpose.

Must Understand

- ▶ Why context switching enables multiprogramming but consumes CPU time, creating the time-slice trade-off.
- ▶ Why PCB register contents are necessary: the OS must restore a process exactly enough for it to resume correctly.
- ▶ Why the OS temporarily retains terminated-process data: a parent may still need the child's termination status via `wait/waitpid`.
- ▶ Why you might use `fork()` without `exec()`: the child can continue executing the same program but follow a different branch / role.

Easy to Confuse

- ▶ **Child `pid == 0`** means `fork()` returned 0 in that process; it does not mean PID 0 is the child's actual identifier.
- ▶ `exec()` **does not spawn another process**; it replaces the current process image.
- ▶ **Terminated does not always mean all bookkeeping vanished immediately**; wait-related status may still be retained.
- ▶ A PCB is **protected kernel metadata**, not ordinary user-process memory.

Source emphasis incorporated: underlined / marked statements on pp.2, 9-10, 12-14, 16-23, 31-37 and highlighted code / annotated answers on pp.40-53; prior discussion clarifications on `pid_t`, child-side `fork()` return value, and the sample `fork-exec-wait` flow are integrated.